Solution Requirements

Phase 2: Requirement Analysis

1. Purpose and Scope

The requirements describe the behaviours the current application is intended to provide. Functional requirements describe user-visible capabilities; non-functional requirements describe qualities such as clarity, reliability and safe configuration. A source file alone does not prove that a requirement has passed end-to-end testing.

2. Functional Requirements

FR-01: Authentication and demo access

Users can sign up and sign in with Supabase Auth. When Supabase is not configured, or the user chooses the one-click demo sign-in, a demo traveller session is created so the whole application can be explored.

FR-02: Guided trip creation

A five-step wizard collects destination, dates, traveller type and count, budget and currency, style, pace, interests, dietary and accessibility needs, and special notes, with validation per step.

FR-03: Generate an itinerary

The backend returns a structured itinerary with a title, day summaries, accommodation suggestion, transit tip and activities (time slot, title, description, start time, duration, estimated cost, location, category, meal type and booking tip).

FR-04: Budget summary

The system computes lodging, food, activity and transit costs, the total estimate and the variance from the target budget, and shows warnings when the plan is over budget.

FR-05: Manage trips

Users can list trips, search by title or destination, filter by travel style, open a trip, edit activities, regenerate a day and delete a trip after confirmation.

FR-06: Profile and settings

Users can edit a profile (name, currency, default style and pace, dietary and accessibility notes) and view system diagnostics and reset demo data on the settings page.

FR-07: Print or save a plan

The itinerary page invokes the browser print dialog using a print stylesheet; the backend can also render a standalone printable HTML page for a trip.

3. Non-Functional Requirements

- Usability: clear labels, a progress stepper, readable errors and responsive layouts.
- Maintainability: pages, components, context, services and utilities are separated; the backend separates API routes, services, schemas and configuration.
- Reliability: if the AI provider or the backend fails, the flow degrades to a fallback generator instead of a blank page.
- Security: provider keys and Supabase service credentials stay on the backend; database access is protected by row-level security.
- Portability: runs in a modern browser; frontend on Vercel and backend on Render.
- Transparency: costs are estimates and the demo banner states when demo mode is active.

4. Acceptance Criteria

A requirement should be marked as passed only after the deployed flow has been tested. For example, generation passes when a valid request produces a visible itinerary and an invalid one produces a clear message; persistence passes when a saved trip can be reloaded, opened and deleted; printing passes when all days appear on the print preview without clipped content.

5. Constraints and Assumptions

The application depends on internet access for the deployed site. Live AI generation needs a valid Gemini or OpenAI key on the backend; live persistence needs a Supabase project. Without them the application runs in demo mode. Costs and suggestions are estimates that the traveller must verify.

6. Traceability

Area	Source location
Routing and layout	frontend/src/App.jsx
Wizard, itinerary, saved trips, edit, profile, settings	frontend/src/pages/*.jsx
API client and demo-mode fallback	frontend/src/services/api.js, mockData.js
Auth and trip state	frontend/src/context/AuthContext.jsx, TripContext.jsx
API routes	backend/app/api/health.py, trips.py, export.py
AI generation and budget	backend/app/services/ai_service.py, budget_service.py
Schemas, configuration, security	backend/app/models/schemas.py, core/config.py, core/security.py
Database schema and RLS	supabase/migrations/001_initial_schema.sql